Guía
de despliegue

Base de Empresas — Fase 1 en Google Cloud y Firebase

OCTUBRE 2026

Contenido

01 Arquitectura

02 Requisitos

03 Primer despliegue

1. Firebase Authentication y la web

2. Infraestructura, API, migraciones y web

3. Usuarios

4. Job del CDR en el servidor Ubuntu

5. Configuración inicial de negocio

6. Verificación

04 Actualizaciones

05 Operación

Backups y restauración

Monitoreo

Rotar los tokens

Seguridad y Ley 18.331

06 Desarrollo local

Tests

Documentos

07 Variables de entorno de la API

08 Problemas frecuentes

Esta guía lleva la Base de Empresas desde el repositorio hasta producción en un proyecto de Google Cloud propio, y explica cómo actualizarla, operarla y desarrollar en local. La fase 0 no se despliega: sus scripts corren en una laptop o en el servidor Ubuntu (ver `fase0/README.md`).

Arquitectura

PIEZA	SERVICIO	QUÉ HACE
Web	Firebase Hosting (`web/public`)	Pantallas en JS sin framework. Reescribe `/api/**` a Cloud Run
Autenticación	Firebase Authentication	Email y clave. La API valida el ID token
API	Cloud Run `empresas-api` (`Dockerfile`)	FastAPI. Reglas de negocio, permisos por rol, auditoría
Base de datos	Cloud SQL Postgres 16 `empresas-pg`	Esquema `empresas`, migraciones en `db/empresas/`
Migraciones	Cloud Run job `empresas-migrar`	Corre `db/migrate.py` con la misma imagen de la API
Tareas diferidas	Cloud Tasks `empresas-tareas`	Preview y confirmación de cargas, ingestas y remapeos del .sav
Archivos	Cloud Storage `<proyecto>-empresas-archivos`	Bases y .sav subidos (privado)
Secretos	Secret Manager	`empresas-database-url`, `empresas-job-token`, `empresas-tarea-token`
Auditoría larga	BigQuery `empresas_auditoria.auditoria`	Export diario a las 5:10 (Cloud Scheduler)
CDR	Servidor Ubuntu de Equipos (`jobs/cdr`)	Job a las 4:00: lee MySQL de uContact y empuja a la API

Todo vive en `southamerica-east1` (São Paulo), salvo el dataset de BigQuery (ver decisiones, D27).

Requisitos

- Un proyecto de Google Cloud con facturación habilitada y permisos de **Owner** para quien despliega (solo la primera vez).
- El mismo proyecto agregado a Firebase (consola de Firebase → *Agregar proyecto* → elegir el proyecto existente).
- En la máquina que despliega: `gcloud` (Google Cloud SDK, incluye `bq`), `firebase-tools` (`npm install -g firebase-tools`), `git` y `openssl`.
- Acceso del servidor Ubuntu a la base MySQL del CDR (`uc`) con un usuario de solo lectura.

```
gcloud auth login
gcloud config set project equipos-base-empresas
firebase login
```

Primer despliegue

1. Firebase Authentication y la web

1. En la consola de Firebase → **Authentication** → *Comenzar* → habilitar **Correo electrónico/contraseña**.
2. En **Configuración del proyecto** → *Tus apps* → agregar una app **Web** y copiar el objeto `firebaseConfig`.
3. Pegarlo en `web/public/js/config.js` (las claves de una app web de Firebase son públicas; el acceso lo controla la API) y dejar `authMode: "firebase"`.
4. Copiar `web/.firebaserc.example` a `web/.firebaserc` con el id del proyecto.
5. En **Authentication** → **Configuración** → **Dominios autorizados** verificar que estén el dominio `*.web.app` y, si se usa, el dominio propio.

2. Infraestructura, API, migraciones y web

Desde la raíz del repositorio:

```
PROYECTO=equipos-base-empresas ADMIN_EMAILS=garaujo@equipos.com.uy ./infra/desplegar.sh todo
```

El script es idempotente (se puede volver a correr) y hace, en orden:

1. **base:** habilita las APIs; crea la cuenta de servicio `empresas-api` con los roles mínimos (Cloud SQL client, Secret Manager accessor, Cloud Tasks enqueuer, BigQuery data editor y job user); crea la instancia de Cloud SQL (Enterprise, 1 vCPU, 3,75 GB, 20 GB SSD con autoincremento, backups diarios a las 6:00 con recuperación a un punto en el tiempo y 14 copias), la base `empresas` y el usuario `empresas_app` con clave aleatoria; guarda en Secret Manager la cadena de conexión y dos tokens aleatorios; crea el bucket privado, la cola de Cloud Tasks, el dataset y la tabla de BigQuery, y el repositorio de Artifact Registry.
2. **api:** construye la imagen con Cloud Build y despliega Cloud Run con `AUTH_MODE=firebase`, `TASKS_MODE=cloudtasks`, el bucket, la cola, la tabla de BigQuery y los secretos. La primera vez completa `API_URL` con la URL que asigna Cloud Run.
3. **migrar:** crea y ejecuta el Cloud Run job `empresas-migrar`, que aplica las migraciones pendientes.
4. **programar:** crea el Cloud Scheduler que exporta la auditoría a BigQuery todos los días a las 5:10.
5. **web:** `firebase deploy --only hosting`.

`ADMIN_EMAILS` lista (separados por coma) los emails que quedan como administradores la primera vez que entran. Después los usuarios se gestionan desde la pantalla **Usuarios**.

3. Usuarios

Cada persona necesita dos cosas con el **mismo email**:

1. Una cuenta en Firebase Authentication (consola → Authentication → *Agregar usuario*, o que el administrador la cree y la persona use "¿Olvidaste tu contraseña?" para elegir su clave).
2. Un registro en la pantalla **Usuarios** de la aplicación con su rol. Sin ese registro, la API responde "no tiene acceso".

4. Job del CDR en el servidor Ubuntu

```
gcloud run services describe empresas-api --region southamerica-east1 --format 'value(status.url)'
gcloud secrets versions access latest --secret empresas-job-token
```

Con esos dos valores, seguir `jobs/cdr/README.md`: clonar el repositorio en `/opt/base-empresas`, crear el entorno virtual, completar `jobs/cdr/cdr_job.env` (MySQL de solo lectura, `API_URL` = URL de Cloud Run, `JOB_TOKEN`), probar con `--dry-run` y habilitar el timer de systemd (4:00, hora de Montevideo).

5. Configuración inicial de negocio

Con un usuario administrador, en la aplicación:

1. **Naturalezas jurídicas:** cargar la hoja "excluidos" del Excel de referencia con `python fase0/cargar_excluidos.py <excel> --api <url> --token <id token>` o marcarlas a mano.
2. **Configuración:** meses de enfriamiento, destinos internos del CDR (salen del experimento A de la fase 0), prefijos de salida de la central, franjas y el formato del CSV de uContact (verificarlo con una importación de prueba).
3. **Alchemer:** cargar el bloque estándar de variables en cada cuestionario (`fase0/alchemer_bloque_estandar.md`).

6. Verificación

```
URL=$(gcloud run services describe empresas-api --region southamerica-east1 --format 'value(status.url)')
curl -s $URL/api/salud                    # {"ok":true}
```

- Entrar a la web con un administrador y ver la pantalla de inicio.
- Hacer una carga chica de prueba y deshacerla.
- Correr el job del CDR a mano (`sudo systemctl start cdr-job.service`) y ver el lote en **Actividad**.

Actualizaciones

CAMBIO	COMANDO
Código de la API	`PROYECTO=… ./infra/desplegar.sh api`
Migración nueva (`db/empresas/NNNN_*.sql`)	`./infra/desplegar.sh api` y después `./infra/desplegar.sh migrar`
Web	`PROYECTO=… ./infra/desplegar.sh web`
Job del CDR	`git pull` en `/opt/base-empresas` (no requiere reiniciar el timer)

Las migraciones se escriben para ser compatibles hacia atrás (agregar columnas o tablas, no borrar) para que la versión anterior de la API siga funcionando mientras corren. Una migración nunca se edita una vez aplicada: se agrega otra.

Volver atrás la API: Cloud Run guarda las revisiones anteriores.

```
gcloud run services update-traffic empresas-api --region southamerica-east1 --to-revisions <revision>=100
```

Volver atrás la web: consola de Firebase → Hosting → historial de versiones → *Revertir*.

Operación

Backups y restauración

Cloud SQL hace un backup diario (6:00) y guarda los registros para recuperación a un punto en el tiempo (7 días). Para probar una restauración sin tocar producción:

```
gcloud sql instances clone empresas-pg empresas-pg-prueba --point-in-time '2026-11-02T10:00:00Z'
```

Se recomienda hacerlo una vez por trimestre (coincide con la auditoría por muestreo del PRD). Los archivos subidos quedan en el bucket; la auditoría, además, en BigQuery.

Monitoreo

- **Errores de la API:** Cloud Run → `empresas-api` → *Registros* (filtrar por severidad ERROR). Cada error no manejado queda con su traza.
- **Tareas diferidas:** si una carga o ingesta queda en error, la pantalla lo muestra con el mensaje; la cola está en Cloud Tasks → `empresas-tareas`.
- **CDR:** la pantalla de inicio muestra la fecha del último lote. En el servidor: `journalctl -u cdr-job.service --since today`.
- **Alertas sugeridas:** en Cloud Monitoring, una política sobre errores 5xx de Cloud Run y otra sobre CPU o almacenamiento de Cloud SQL por encima del 80%.

Rotar los tokens

```
openssl rand -hex 32 | gcloud secrets versions add empresas-job-token --data-file=-
PROYECTO=… ./infra/desplegar.sh api         # toma la versión nueva
PROYECTO=… ./infra/desplegar.sh programar   # actualiza el Cloud Scheduler
```

Después actualizar `JOB_TOKEN` en `jobs/cdr/cdr_job.env` del servidor Ubuntu. El token de tareas se rota igual (`empresas-tarea-token`) y solo requiere redesplegar la API.

Seguridad y Ley 18.331

- La API rechaza todo pedido sin token válido de Firebase o de un email no registrado. Los permisos se validan en el backend.
- El bucket es privado (acceso uniforme, prevención de acceso público).
- La base no tiene IP pública habilitada para usuarios: Cloud Run entra por el conector de Cloud SQL.
- Exportar contactos personales requiere el permiso específico del usuario y queda auditado.
- Pendiente legal antes de operar: inscripción de la base ante la URCDP y plazo de conservación de contactos sin uso.

Desarrollo local

Requisitos: Python 3.12+, Postgres 16 y Node (solo para capturas y PDFs).

```
pip install -r api/requirements-dev.txt
createdb empresas_dev
DATABASE_URL=postgresql://postgres@localhost/empresas_dev python db/migrate.py

# Datos de demostración (ficticios) en una base vacía
DATABASE_URL=postgresql://postgres@localhost/empresas_dev python api/demo/sembrar_demo.py

# API + web en http://localhost:8080, entrando con un email de la demo (modo dev)
cd api && DATABASE_URL=postgresql://postgres@localhost/empresas_dev AUTH_MODE=dev SERVE_WEB=1 \
  uvicorn app.main:app --port 8080 --reload
```

En `localhost`, si `config.js` no tiene Firebase configurado, la web entra en modo dev: se ingresa con un email registrado (por ejemplo `admin@equipos.com.uy` de la demo) sin clave. **Nunca** desplegar con `AUTH_MODE=dev`.

Tests

```
createdb empresas_test
cd api && TEST_DATABASE_URL=postgresql://postgres@localhost/empresas_test python -m pytest -q
cd fase0 && python -m pytest -q
cd jobs/cdr && python -m pytest -q
```

Los tests de la API usan Postgres real: cada test migra un esquema limpio. Cubren los criterios de aceptación de R1.1 a R1.13, incluido el control de que ninguna empresa excluida, en no llamar o en enfriamiento entra a una muestra ni a un CSV, y que toda muestra se reproduce con su semilla.

Documentos

```
node docs/build/capturas.js http://localhost:8080 docs/manual/img <rut> api/demo/clima_parte_a_ola1.sav
<base.csv>
python docs/build/generar_pdfs.py
```

Variables de entorno de la API

VARIABLE	PRODUCCIÓN	DESCRIPCIÓN
`DATABASE_URL`	secreto	`postgresql://usuario:clave@/empresas?host=/cloudsql/PROYECTO:REGION:INSTANCIA`
`AUTH_MODE`	`firebase`	`dev` solo en local y tests
`FIREBASE_PROJECT_ID`	id del proyecto	Para validar los tokens
`ADMIN_EMAILS`	emails	Administradores iniciales
`JOB_TOKEN`, `TAREA_TOKEN`	secretos	Job del CDR y export de auditoría; Cloud Tasks
`TASKS_MODE`	`cloudtasks`	`background` en local, `inline` en tests
`TASKS_QUEUE`	`projects/…/queues/empresas-tareas`	Cola de Cloud Tasks
`API_URL`	URL de Cloud Run	A dónde llama Cloud Tasks

VARIABLE	PRODUCCIÓN	DESCRIPCIÓN
`STORAGE_BUCKET`	bucket	Sin bucket, los archivos van a `STORAGE_DIR`
`BQ_TABLA_AUDITORIA`	`proyecto.empresas_auditoria.auditoria`	Export diario
`CORS_ORIGINS`	vacío	Solo si la web se sirve desde otro dominio
`APP_TZ`	`America/Montevideo`	Zona horaria de franjas y fechas

Problemas frecuentes

SÍNTOMA	CAUSA PROBABLE	QUÉ HACER
"no tiene acceso a la Base de Empresas"	El email no está en **Usuarios** o está inactivo	Agregarlo con su rol
"Token inválido"	Reloj del equipo desfasado o sesión vencida	Cerrar sesión y volver a entrar
Pantalla "Falta configurar Firebase"	`config.js` con valores de ejemplo	Completar `firebaseConfig` y redesplegar la web
Una carga queda "analizando" mucho tiempo	La tarea falló o la cola está pausada	Ver Cloud Tasks y los registros de Cloud Run; la pantalla muestra el error
El CSV sale con menos renglones de lo esperado	Ítems sin teléfono discable o ya completos en la parte	El resumen de la exportación lo detalla
El job del CDR termina con código 1	La API no respondió	Se reintenta en la próxima corrida desde el último lote enviado
"Archivo demasiado grande"	Más de 30 MB	Pasar el Excel a CSV o partirlo